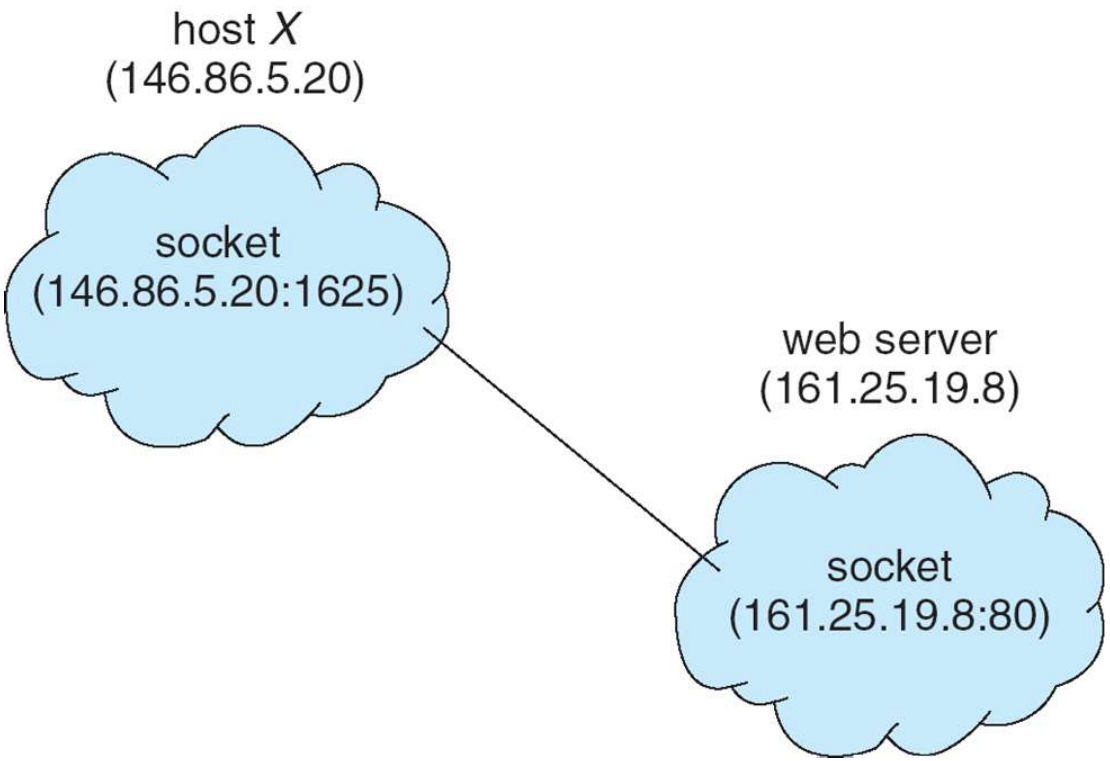
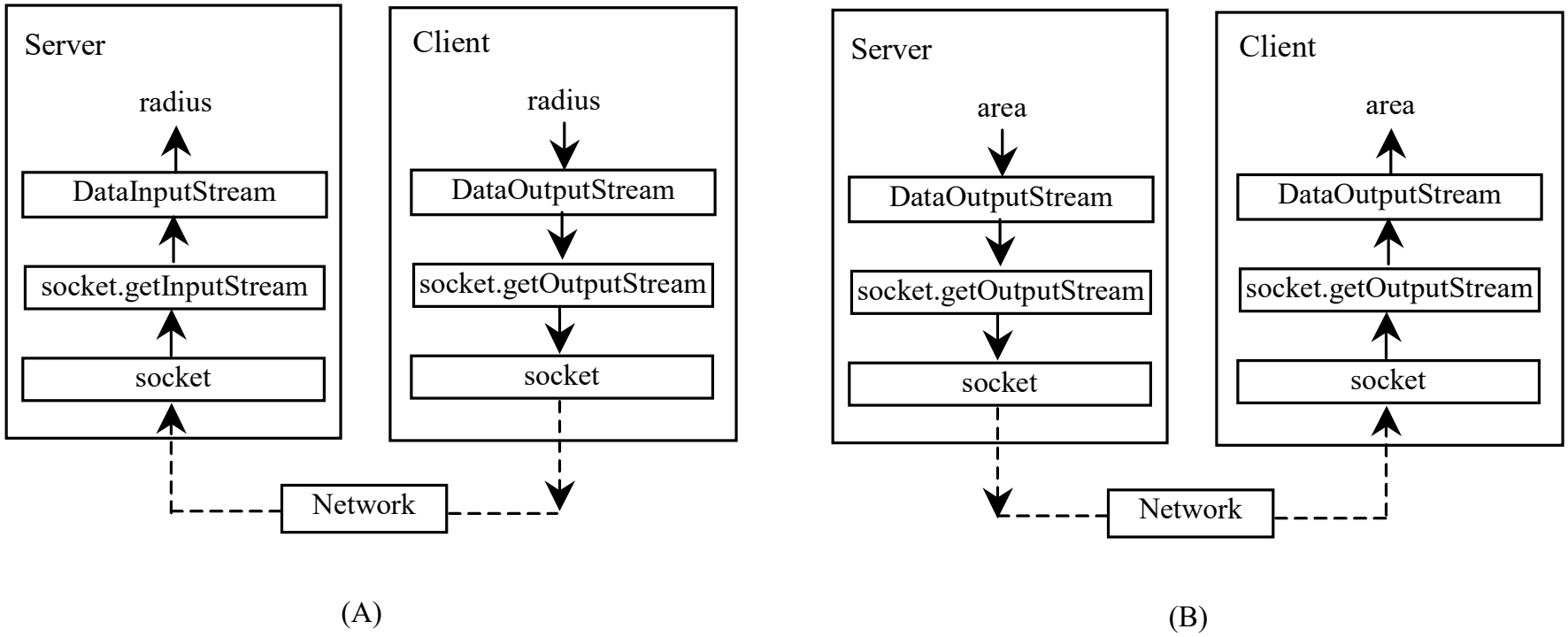
Sockets

Socket

- A **socket** is defined as an endpoint for communication.
- Combination of IP address and **port.**



An application with a server and a remote client to compute an area of a circle:



DataInputStream reads bytes from the stream and converts them into appropriate primitive-type values or strings.
DataOutputStream converts primitive-type values or strings into bytes and outputs the bytes to the stream.
DataInputStream/DataOutputStream are created using the following constructors:
public DataInputStream(InputStream instream)
public DataOutputStream(OutputStream outstream)

Server

```
import java.net.*;
import java.io.*;
public class Server
    {
      public static void main(String[] a)
        {
          try
           {
                ServerSocket serversocket = new ServerSocket(8000);   //create a server socket
                System.out.println("Server Started");
                Socket socket = serversocket.accept();   //listen for connection request
                DataInputStream ip = new DataInputStream(socket.getInputStream()); //create data input & output streams – input from client
                 DataOutputStream op = new DataOutputStream(socket.getOutputStream()); //create data input & output streams – output to client
                 double radius = ip.readDouble();   //read radius
                 System.out.println("Radius received:" + radius);
                double area = radius * radius * Math.PI;
                 System.out.println("Area Computed:" + area);
                op.writeDouble(area);    //write area
           }
         catch(IOException ex)
           {      System.err.println(ex);  }
      } }
```

Client

```
import java.net.*;
import java.io.*;
public class Client
{
   public static void main ( String[] a)
    {
       double radius = 10.0;
        try
            {
                Socket clients = new Socket("localhost",8000);    //create a socket to connect to the server
                 DataInputStream ip = new DataInputStream(clients.getInputStream()); //create an input stream to receive data from the server
                 DataOutputStream op = new DataOutputStream(clients.getOutputStream()); //create an output stream to send data to the server
                 op.writeDouble(radius);  //send radius to server
                 op.flush();
                double area = ip.readDouble(); //get area from server
                 System.out.println("Area is:" + area);
                 clients.close();  //close the socket
            }
    catch(IOException ex)
  {
     System.err.println(ex);
  }
} }
```

JAVA Sockets

- **Package java.net**
  - **java.net.Socket**
    - Implements client sockets (also called just “sockets”).
    - Constructor and Methods
      - Socket(String host, int port): Creates a stream socket and connects it to the specified port number on the named host.
      - InputStream getInputStream()
      - OutputStream getOutputStream()
      - close()

  - **java.net.ServerSocket**
    - Implements server sockets.
    - Waits for requests to come in over the network.
    - Performs some operation based on the request.
    - Constructor and Methods
      - ServerSocket(int port)
      - accept(): Listens for a connection to be made to this socket and accepts it. This method blocks until a connection is made.

Implementation in Java

1. Open the Server Socket:

ServerSocket server = new ServerSocket(PORT);

2. Wait for the Client Request:

Socket client = server.accept(); //socket to connect to the client

Socket socket = new Socket(ip,PORT); //connect to the server – used by client

3. Create I/O streams for communicating to the client

DataInputStream is = new DataInputStream(client.getInputStream());

DataOutputStream os = new DataOutputStream(client.getOutputStream());

4. Perform communication with client

Receive from client: *String line = is.readLine();*

Send to client: *os.writeBytes("Hello \n");*

5. Close socket:

client.close();